

Accelerometer Algorithm Inputs — What Signal Each Stage Matches Against

May 11, 2026

Purpose. Quick reference for what every accelerometer-only algorithm in this repo consumes as input, what it does in plain words, and how the trapezoid match in *segmentation* differs from the trapezoid match in *prediction*. Barometer paths are out of scope.

1 Common signal definitions

Every accelerometer algorithm starts from the raw 3-axis device-frame sample (a_x, a_y, a_z) at sampling rate f_s (typically 100 Hz). From it we derive three candidate scalar signals:

- $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$
Gravity *included*. Orientation-blind.
Used by: the GT editor (`src/data/gt_editor.py:227`) for visual annotation only. **No active algorithm consumes $|a|$.**
- a_z
Device-frame z -axis. **No active algorithm consumes a_z .**
- $a_{\text{vert}} = (a - \hat{g}) \cdot \frac{\hat{g}}{\|\hat{g}\|}$
Signed scalar projection of the acceleration vector onto the estimated gravity direction $\hat{g}$. Positive = up.
This is the signal every active accelerometer algorithm uses.

The gravity vector $\hat{g}$ is estimated from a stationary window: `estimate_gravity_stationary` in `src/utils/accelerometer_utils.py`. In segmentation the window is the first 0.5s of the buffer (`...template_match/fit_elevator_parameters/common.py:136`). In prediction the pre/post ride stationary windows are preferred, falling back to in-ride estimation, then to $|a|$ as a last resort (`trapezoid_accel/estimator.py:118-170`).

After projection the signal is detrended (slow rolling mean) and smoothed (rolling mean over `smooth_sec=0.4s`) to remove DC bias from orientation drift and hand-tremor:

$$a_{\text{vert}}^{(\text{smooth})}(t) = \text{RollMean}(a_{\text{vert}}(t) - \text{RollMean}_{\text{slow}}(a_{\text{vert}}(t)), w_{\text{smooth}}).$$

2 Segmentation — accelerometer path

Single active algorithm: **trapezoid matched-filter template match**, dispatched as `SegmentAlgorithm.ACC_TEMPLATE_MATCH`.

Files: `src/segmentation/algorithms/accelerometer_only/template_match/—matcher.py` (legacy learned-template path) and `check_grid_across_signal/detect.py` (active synthetic-grid path, wired to the editor UI).

Input. Smoothed a_{vert} over the *entire session* (`matcher.py:118`, `detect.py` via the shared preprocessing in `fit_elevator_parameters/common.py:135-144`). The matcher also computes an LPF velocity v_{lpf} by integrating a_{vert} , and the two scores are linearly fused (`matcher.py:125-127`); the active grid detector uses a_{vert} alone.

Template. Synthetic trapezoid kernel parameterised by half-width W and flat-fraction $f \in [0, 1]$:

$$\text{tpl}(t; t_c, W, f) = \begin{cases} 1 & |t - t_c| \leq fW \quad (\text{flat top}) \\ \frac{W - |t - t_c|}{W(1 - f)} & fW < |t - t_c| \leq W \quad (\text{linear ramp}) \\ 0 & |t - t_c| > W. \end{cases}$$

Grid: $30 \times 15 = 450$ templates over $W \in [0.3, 3.0]$ s, $f \in [0.05, 0.80]$, plus the $f = 0$ triangle row (`detect.py:130-143`).

What it does (problem type: *detection*). Slide every template across the whole signal using Pearson normalised cross-correlation (`ncc_slide`, `matcher.py:33-54`). Reduce the per-sample argmax across (W, f) to a peak score, then:

1. **Peak-pick** on unsigned R^2 with threshold `r2_peak_thresh=0.40` and a minimum matched amplitude $|A| \geq 0.20$ m/s².
2. **NMS dedup** within `nms_radius_s=1.0` s and same-sign suppression within 5 s.
3. **Pair** a positive (take-off) peak with a later negative (landing) peak, accept if their shared-shape joint $R^2 \geq 0.90$, the pair $|A| \geq 0.30$ m/s², and the plateau between the lobes is quiet (`quiet_middle_ratio=0.5`).

Output. A list of ride intervals $(t_{\text{start}}, t_{\text{end}}, \text{direction})$. No height value is produced.

3 Prediction — accelerometer paths

Two accelerometer-only predictors share the `predict_segment(ride, pre, post) -> PredictionOutput` contract dispatched by `Predictor` in `src/prediction/algorithms/predictor.py:29-43`.

3.1 ZUPT double-integration

File: `src/prediction/algorithms/accelerometer_only/zupt_accel/estimator.py`.

Input. a_{vert} over the *ride window*, plus the pre/post stationary windows used to estimate $\hat{g}$.

What it does.

1. Detect the active-motion sub-window by thresholding $|a_{\text{vert}}|$ (`_find_motion_window`, `estimator.py:50-65`).
2. Zero-force $a_{\text{vert}}(t) = 0$ outside that sub-window (`_zupt_integrate_windowed`, `estimator.py:68-79`).
3. Double-integrate $a_{\text{vert}} \rightarrow v \rightarrow h$, applying linear-drift correction so v returns to zero at the boundaries.

Trapezoid involvement. **None.** ZUPT does not match any template; it is a pure double-integration with zero-velocity-update bookends.

Output. Signed Δh in metres plus a theoretical σ from white-noise integration bounds.

3.2 Trapezoid pulse-pair fit

File: `src/prediction/algorithms/accelerometer_only/trapezoid_accel/estimator.py`;
kernel + grid search in `pulse_pair.py`.

Input. Detrended + smoothed a_{vert} over the *ride window* (`estimator.py:190-196`): same projection, same `smooth_sec=0.4s` rolling mean.

What it does (problem type: *parameter estimation*). Searches the same (W, f) trapezoid grid as segmentation (`pulse_pair.py:42`, 30×15). For every grid point and every candidate centre pair (t_{c1}, t_{c2}) it solves least-squares for a *shared* amplitude $|A|$ across the two lobes and scores by the joint $R^2 = \frac{1}{2}(R_1^2 + R_2^2)$ (`pulse_pair.py:42-62`). The argmax across (W, f, t_{c1}, t_{c2}) is kept. A second 4-parameter *joined-pulse* fit ($\Delta t_c = 2W$ forced) is run in parallel; the better joint- R^2 wins (`estimator.py:213-254`). A ZUPT fallback kicks in if both matched-filter fits collapse.

Output. Closed-form

$$\Delta h = \text{sign} \cdot |A| \cdot W \cdot (1 + f) \cdot (t_{c2} - t_{c1})$$

(`estimator.py:330-339`), plus a delta-method theoretical σ and a conformal-calibrated 90% CI half-width.

4 Segmentation vs. prediction — same trapezoid, different question

Both stages match the same trapezoid template family against the same projected signal a_{vert} . The difference is what they do with the match.

	Segmentation	Prediction (trapezoid pulse-pair)
Input scope	smoothed a_{vert} over the whole session	detrended+smoothed a_{vert} over one ride window
Template family	synthetic trapezoid (W, f) grid + $f=0$ triangle row	same (W, f) grid, shared-shape constraint across the lobe pair
Scoring	sliding Pearson NCC, per-sample peak score	exhaustive LS amplitude per (W, f, t_{c1}, t_{c2}) , joint R^2
What is found	peak <i>locations</i> (when does a ride happen?)	best-fit <i>parameters</i> $(W, f, A , t_{c1}, t_{c2})$
Problem class	detection	parameter estimation
Output	ride intervals (t_s, t_e, dir)	Δh in metres + CI half-width
Filters / gates	peak $R^2 \geq 0.40$, NMS, pair $R^2 \geq 0.90$, quiet-middle	quality score, R^2 thresholds by ride length, anchor-ratio, Δt_c floors

The trapezoid kernel itself lives in a single shared utility (`src/utils/trapezoid_template.py`, re-exported by both stages) so the template parametrisation is identical by construction.

5 One-line summary

All active accelerometer algorithms — the segmentation matched-filter, the ZUPT predictor, and the trapezoid pulse-pair predictor — consume a_{vert} (gravity-projected vertical acceleration), not $|a|$. The trapezoid template appears in both stages, but with different roles: segmentation uses it as a *detector* (sliding NCC over the whole session to find rides), and prediction uses it as an *estimator* (exhaustive LS fit inside a single ride to recover Δh).

Implication for editor graphs. The GT editor (`src/data/gt_editor.py`) currently plots $|a|$, which no active algorithm uses. The segmentation editor (`src/segmentation/algorithms/editor.py:118-121`) and the Streamlit segmentation step (`src/pipelines/streamlit/step3_segmentation.py:199-209`) both already plot a_{vert} (with its smoothed overlay), which is the signal the matched filter actually scores against. Harmonising the three to display a_{vert} is a follow-up; this report is the reference for the decision.